

`math.floor(len(cabinet)/2)`: to refer to the entire first half or the entire second half of the original cabinet, respectively. You can call this function with any one- or two-element cabinet—for example, `mergesort_two_elements([3,1])`—and see that it successfully returns a sorted cabinet.

Next, let's write a function that can sort a list that has four elements. If we split a four-element list into two sublists, each sublist will have two elements. We could follow our merging algorithm to combine these lists. However, recall that our merging algorithm is designed to combine two already sorted lists. These two lists may not be sorted, so using our merging algorithm will not successfully sort them. However, each of our sublists has only two elements, and we just wrote a function that can perform merge sort on lists with two elements. So we can split our four-element list into two sublists, call our merge sort function that works on two-element lists on each of those sublists, and then merge the two sorted lists together to get a sorted result with four elements. This Python function accomplishes that:

```
def mergesort_four_elements(cabinet):
    newcabinet = []
    if(len(cabinet) == 1):
        newcabinet = cabinet
    else:
        left = mergesort_two_elements(cabinet[:math.floor(len(cabinet)/2)])
        right = mergesort_two_elements(cabinet[math.floor(len(cabinet)/2):])
        newcabinet = merging(left,right)
    return(newcabinet)

cabinet = [2,6,4,1]
newcabinet = mergesort_four_elements(cabinet)
```

We could continue writing these functions to work on successively larger lists. But the breakthrough comes when we realize that we can collapse that whole process by using recursion. Consider the function in [Listing 4-5](#), and compare it to the preceding `mergesort_four_elements()` function.

```
def mergesort(cabinet):
    newcabinet = []
    if(len(cabinet) == 1):
        newcabinet = cabinet
    else:
        ❶ left = mergesort(cabinet[:math.floor(len(cabinet)/2)])
        ❷ right = mergesort(cabinet[math.floor(len(cabinet)/2):])
```
